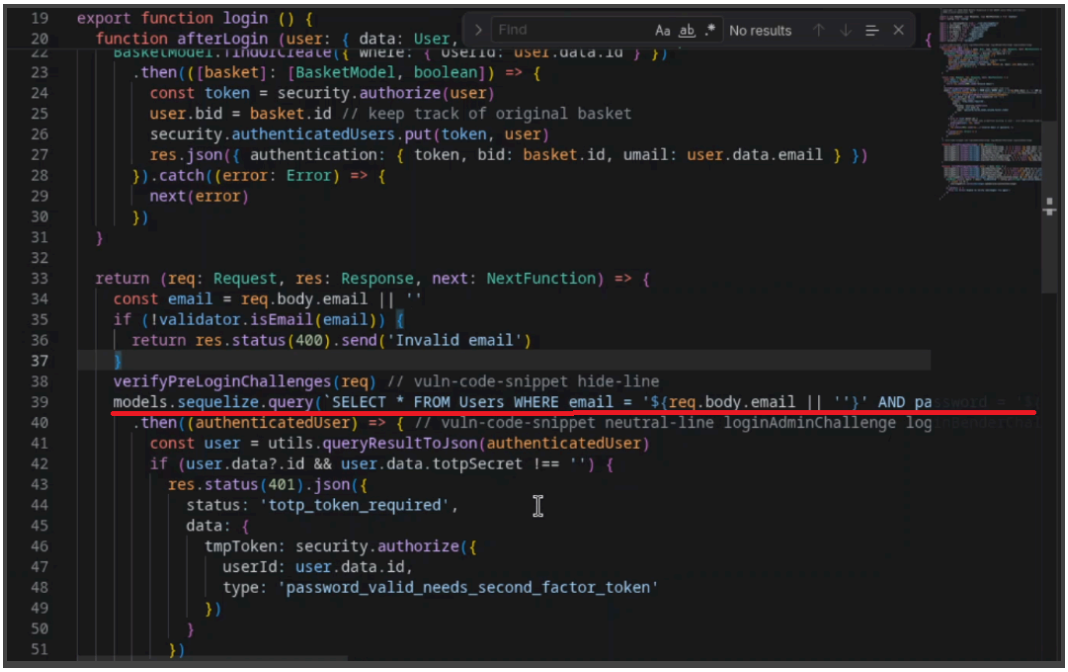
Week 2 – Blue Teaming

1.0 Fixing Vulnerabilities

1.1 Identifying the SQL Injection (SQLi) Vulnerability

In the previous week, we discussed what an SQL Injection was, and found it as a vulnerability within the local host. From the previous week, we understood that the line that was acting problematic was:



(Figure 1.1.1)

The full text for that line contained the following:

models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })

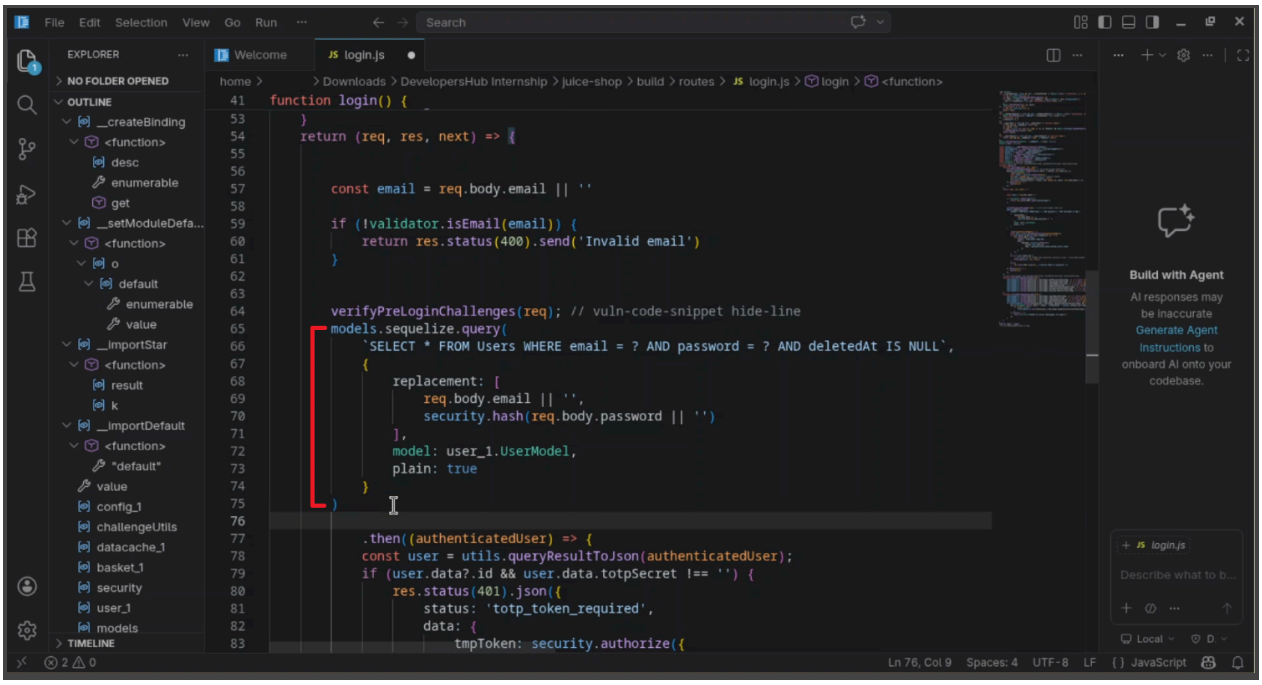
The reason why this line is vulnerable is:

'${req.body.email || ''}'

This is because it directly takes input from the user and executes whatever is entered within.

1.2 Eradicating the Vulnerability

Now, this line should be instead be replaced with:



(Figure 1.2.1)

```
models.sequelize.query(
        'SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS
        NULL',
        {
                replacements: [
                        req.body.email || '',
                        security.hash(req.body.password || '')
                ],
                model: UserModel,
                plain: true
        }
)
```

I also had to initialise *"email"* written above the conditional validator statement:

const email = req.body.email || ''

2.0 Sanitizing and Validating Inputs

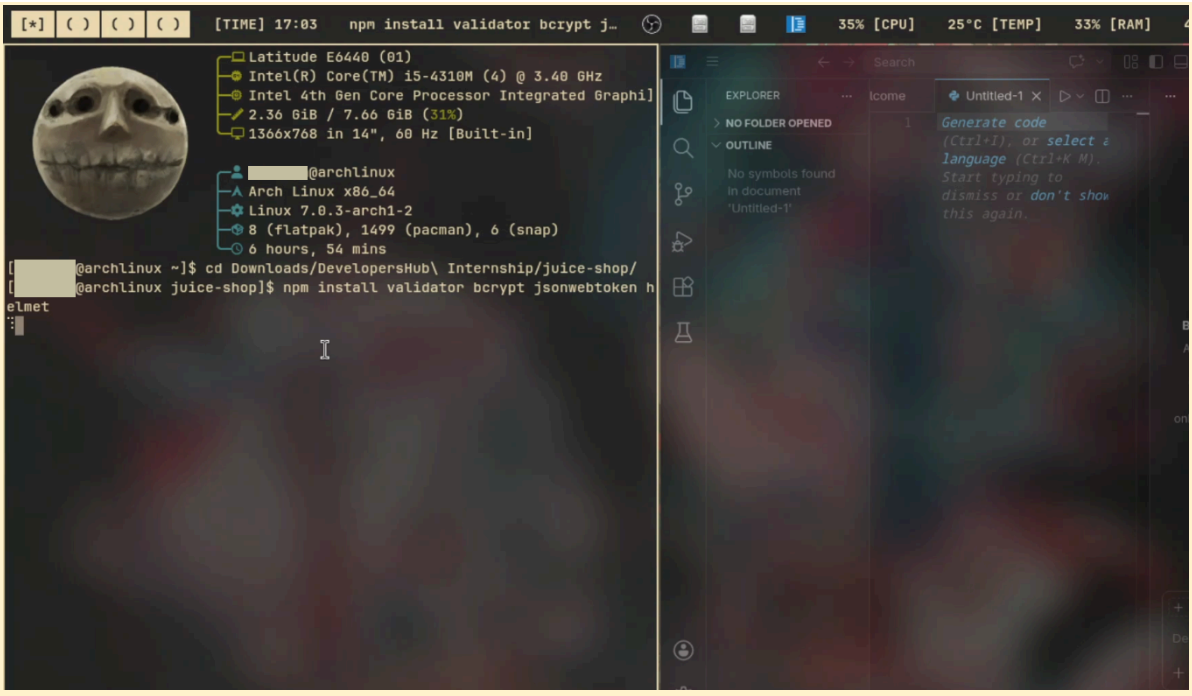
2.1 Validating Inputs

Validating input means checking whether the data provided by a user is in the correct format and meets expected rules before it is processed. For example, an email field should follow a valid email structure, and a password might need to meet certain length or complexity requirements. If the input does not meet these rules, it is rejected. Validation ensures that only acceptable and properly structured data is allowed into the system.

2.1.1 Installation of all Relevant Libraries

In order to add a validator, I opened the terminal and installed the validator library (along with all the other imports needed to finish the task for week 2):

npm install validator bcrypt jsonwebtoken helmet

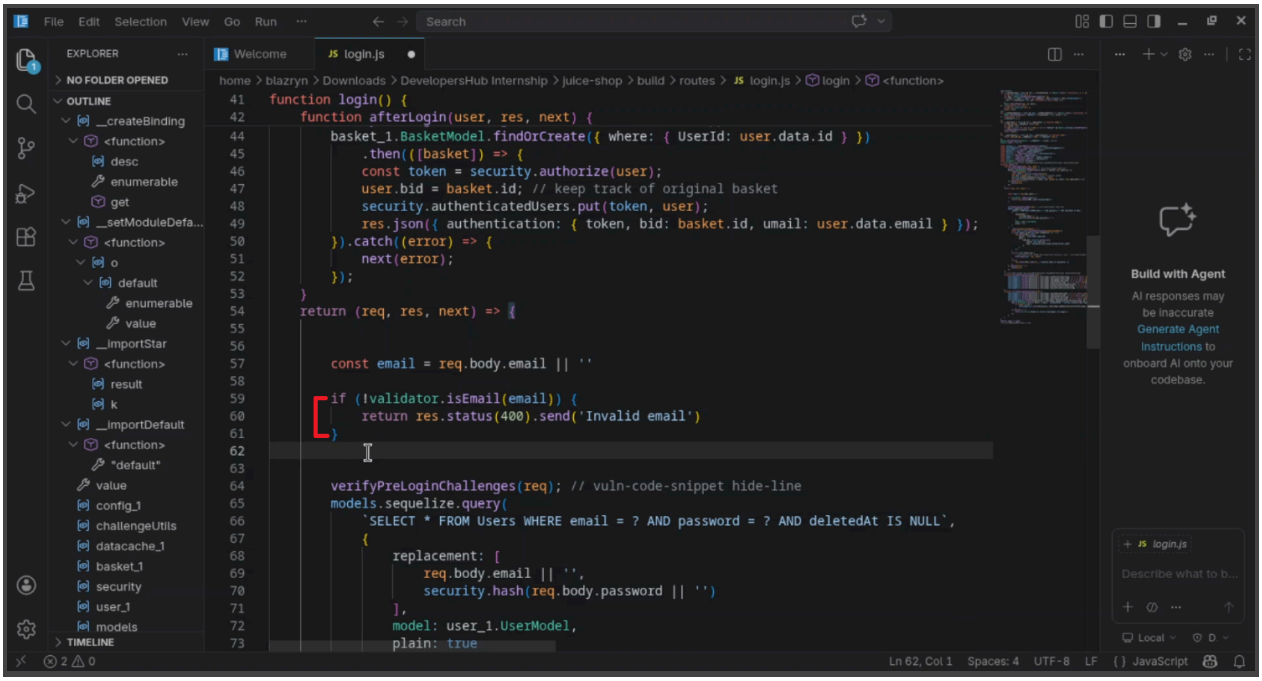


(Figure 2.1.1)

2.1.2 Adding a Validator

Since all the required applications had now been imported, I started with the first task at hand - the validator. I opened Code (The Open Source version for VS code) and opened the localhost /build/routes files. I located login.js in the list of routes and wrote the following code:

Figure 2.1 shows the code I wrote to validate inputs. The red [bracket text shows the validator that I added.



(Figure 2.1.2)

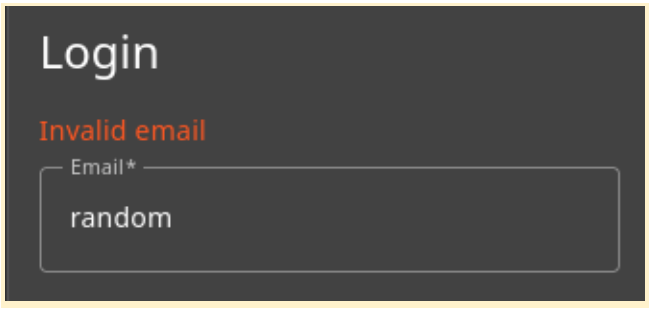
If (!validator.isEmail(email)) {
return res.status(400).send('Invalid email')
}

2.1.3 Testing Validator

This makes it so that invalid email formats cannot be entered. An example can be entering values such as "random" as email prompts "Invalid email", which means that the validator prompted an error. The validator was also imported using:

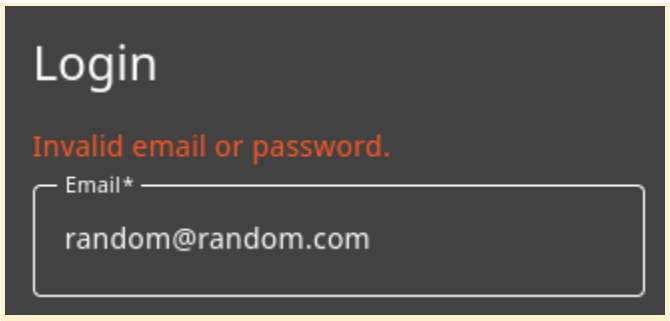
const validator = require('validator');

Figure 2.1.3 Shows how using values such as “random” (instead of the correct format of writing an email e.g “random@random.com”) prompts “Invalid email” as expected.



(Figure 2.1.3)

Figure 2.1.4 The usual error message would display “Invalid email or password”:



(Figure 2.1.4)

2.2 Sanitizing Inputs

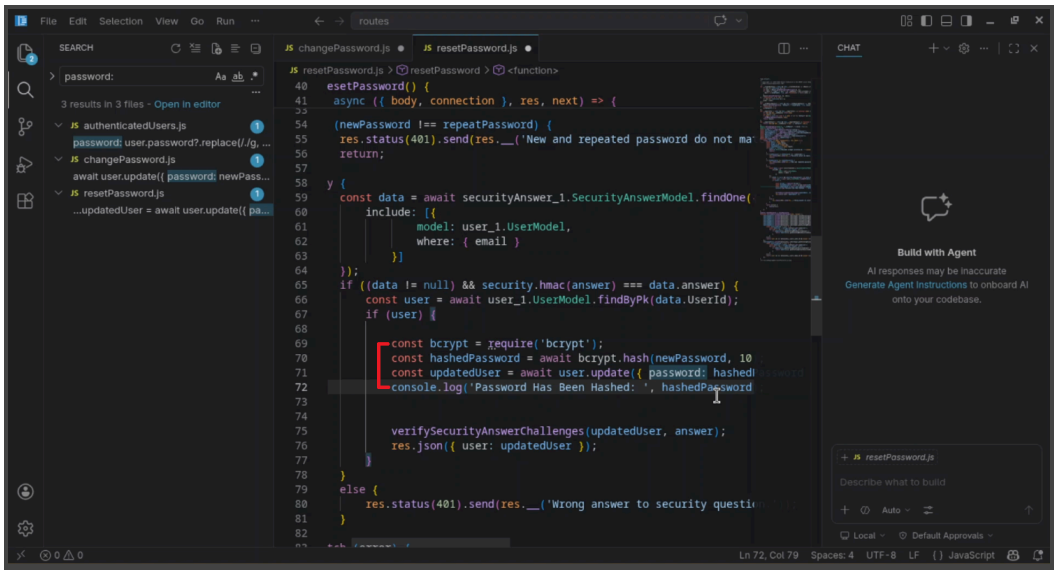
Sanitizing input means cleaning or modifying user input to remove or neutralize potentially harmful content before it is used by the application. This may include removing special characters, escaping HTML tags, or converting unsafe input into a safe format. The goal is to prevent malicious data from being executed or misinterpreted by the system.

2.2.1 Adding Bcrypt (Sanitization)

Since bcrypt had already been installed through npm install, all there is left is to open files and find areas where bcrypt can be used. I used bcrypt within /build/routes/resetPasswords as an example. I proceeded to write the following lines of code:

```
const bcrypt = require('bcrypt');
const hashedPassword = await bcrypt.hash(password, 10);
const updatedUser = await user.update({ password: hashedPassword });
console.log('Password Has Been Hashed: ', hashedPassword);
```

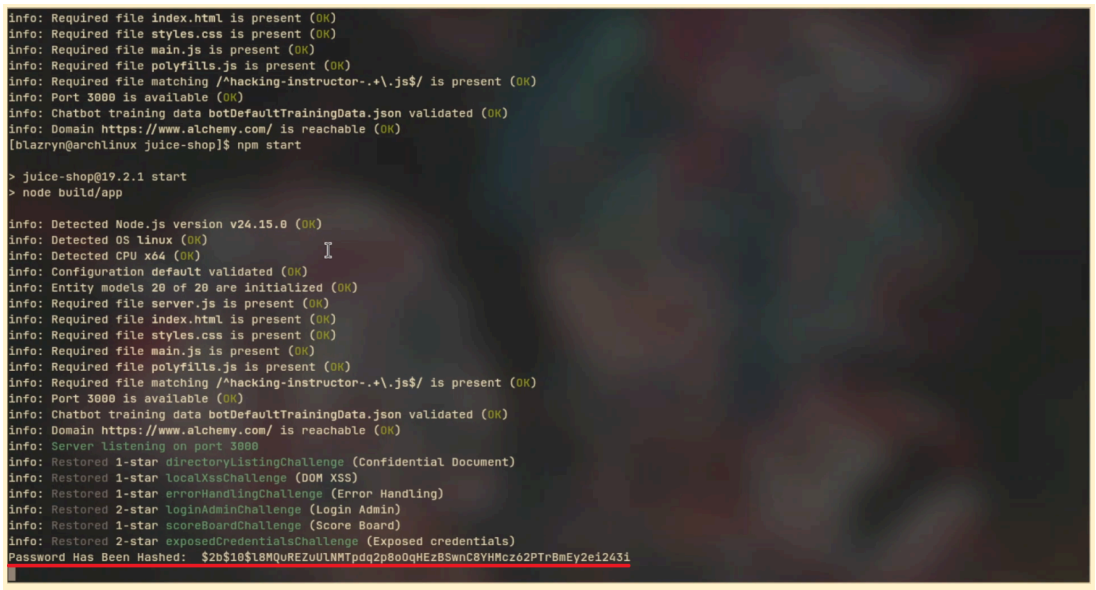
Figure 2.2.1 Shows code written within the red [bracket:



(Figure 2.2.1)

2.2.1 Testing Bcrypt

The *console.log('Password Has Been Hashed: ', hashedPassword);* line is what acts as an indicator that the password has been hashed and hence bcrypt is working. Which I tested this and it worked, since I received it's output within my terminal (shown by the red underline):



(Figure 2.2.2)

3.0 Utilising Helmet

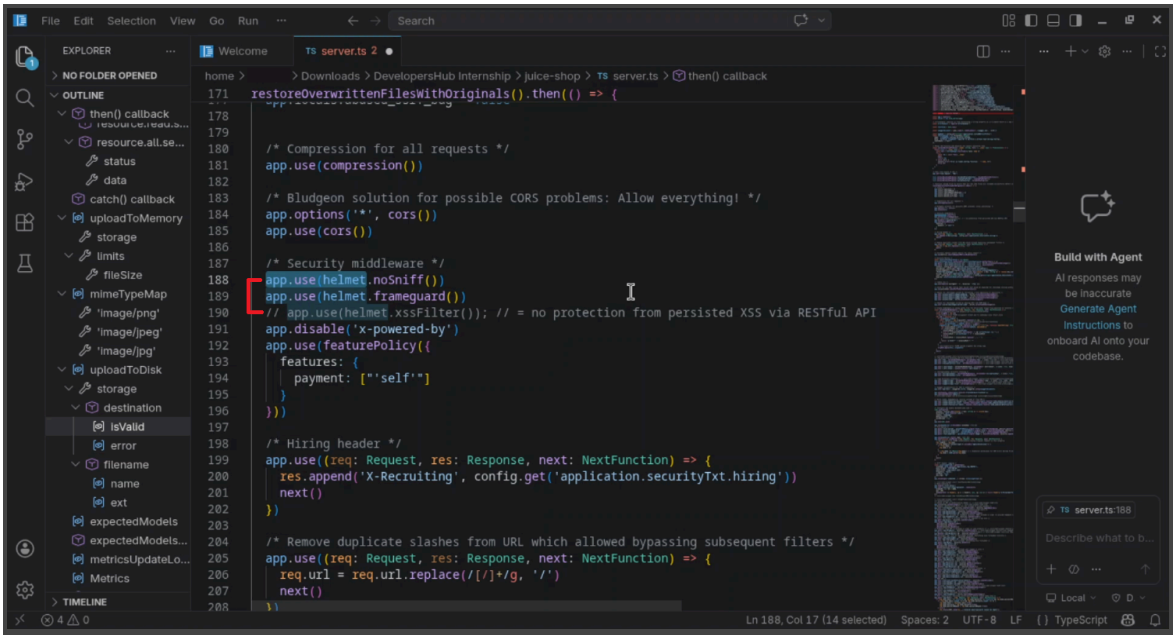
3.1 What Helmet Does

Helmet is a security middleware library used in Node.js and Express applications to help protect web applications by setting secure HTTP headers automatically. These headers improve browser-side security and reduce the risk of several common web-based attacks.

3.2 Helmet Already Added

I reviewed the existing Helmet configuration already present in the application, and observed that helmet was a preexisting import within OWASP juiceshop. I found that there were three lines in particular that were already written in server.ts:

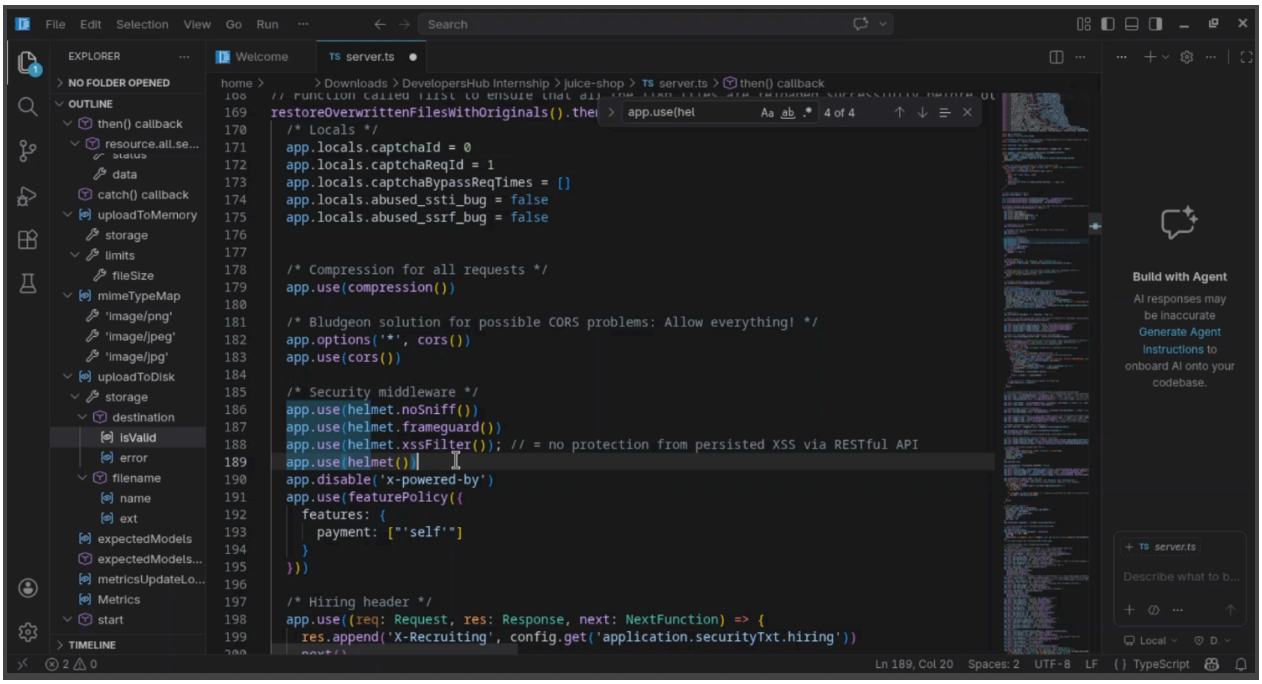
app.use(helmet.noSniff())
app.use(helmet.frameguard())
//app.use(helmet.xssFilter()); // = no protection from persisted XSS via RESTful API



(Figure 3.1)

Hence, to simply remove the XSS Filter, I uncommented the 3rd line by removing the // lines. I also added the following line below so that helmet could fully be used instead of specific utils:

app.use(helmet())



(Figure 3.2)

4.0 Json Web Token (JWT)

4.1 What JWT Does

The objective of this task was to investigate and understand how token-based authentication works in web applications and determine whether JSON Web Tokens (JWT) could be integrated into the OWASP Juice Shop authentication system. The assignment specifically required implementing token-based authentication using the *jsonwebtoken* library.

JWT helps applications verify user identity securely without requiring users to log in repeatedly for every request. The token is digitally signed using a secret key, which helps ensure that the token cannot easily be modified or forged by attackers. If someone attempts to change the contents of the token, the server can detect that the signature is invalid.

4.2 Adding Json Web Token (JWT)

To import JWT, I had to add the following line:

const jwt = require('jsonwebtoken')

After it would be imported, I would add these lines to initiate the JWT:

```
const token = jwt.sign(
        { id: user.data.id },
        "secret-key",
        { expiresIn: "1h"}
)
```

4.2 Authentication Already Added

After investigating the application structure, it was discovered that OWASP Juice Shop already used its own internal authentication system through:

security.authorize(user)

Instead of replacing the built-in authentication mechanism entirely, the existing system was retained.